>_ MCMP113 · Week 15

Final Exam Review & Practice Final

Cumulative cheat sheet · review questions · practice final · problem set

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

Week 15 is the final-exam week, so these notes contain no new material. They have three parts. **(1) Review**: a three-page cumulative cheat sheet (Section 2) and the fourteen practice questions of the lecture with worked answers (Section 4). **(2) Practice Final Exam** (Section 5): 120 minutes, 100 points, eight questions of the same types as the real exam. Take it *once*, on paper, with a timer. **(3) Problem set** (Section 6): fourteen extra problems with difficulty stars. Every program mentioned here is in the Week 15 code folder (`15_01` ... `15_15`) and has been compiled (0 errors, 0 warnings) and run; every output shown is real output. Programs that read files create those files themselves first. Run `15_01_PredictChecker` only *after* you have answered the predict and trace questions yourself. The final is cumulative: for Weeks 1–7 also use the Week 8 package (cheat sheet, practice midterm).

Contents

1 The final exam **2**
1.1 Learning outcomes checked by the final . . . 2
1.2 Logistics . . . 3
1.3 Question types and how to answer them . . . 3

2 Cumulative cheat sheet **4**
2.1 C# syntax summary . . . 4
2.2 Array algorithms at a glance . . . 5
2.3 Parameters, errors and file methods . . . 5

3 Common mistakes **6**

4 Review questions from the lecture **6**
4.1 Q1–Q4: multiple choice . . . 6
4.2 Q5: arrays . . . 7
4.3 Q6: bubble sort, pass by pass . . . 8
4.4 Q7: binary search trace . . . 8
4.5 Q8: nested loops on a matrix . . . 9
4.6 Q9: value, `ref` and array parameters . . . 9
4.7 Q10: recursion . . . 10

4.8 Q11: spot the compile errors . . . 10
4.9 Q12: find the bugs . . . 11
4.10 Q13: write the methods . . . 12
4.11 Q14: a CSV file program . . . 14

5 Practice Final Exam **16**
Question 1: Multiple choice . . . 16
Question 2: Predict the output . . . 17
Question 3: Trace a sort . . . 18
Question 4: Matrices . . . 18
Question 5: Methods, parameters and scope . . . 19
Question 6: Find and fix the errors . . . 20
Question 7: Write a program (arrays and methods) . . . 20
Question 8: Write a program (files) . . . 21

6 Additional problem set **21**

7 Summary **23**

References and further reading **24**

>_ 1 The final exam

1.1 Learning outcomes checked by the final

After Weeks 1–14 you should be able to:

1. express an algorithm in pseudocode and as a flowchart, and trace it with a trace table;
2. write correct C# with variables, operators, selection and loops (Weeks 1–7, see the Week 8 notes);
3. declare, fill, traverse and copy one-dimensional arrays, and use the standard array patterns (sum, maximum with index, counting, frequency array, reverse, filter);
4. trace and write linear search, binary search, bubble sort and selection sort, and compare their number of steps;
5. process matrices (`int[,]`) with nested loops: row and column sums, diagonals, transpose, output as a grid;
6. define and call methods, predict the effect of value, array, `ref` and `out` parameters, and return one or several values;
7. explain `return`, scope and lifetime; recognise the compiler errors CS0103, CS0136, CS0161, CS0165, CS0029, CS1503 and CS8421; trace short recursive methods;
8. read and write text files (`File` methods, `StreamReader`/`StreamWriter` with `using`), parse CSV lines with `Split`, and handle `FileNotFoundException` and `FormatException`.

1.2 Logistics

Items marked **[placeholder]** are not decided in these materials; the official information is posted on the course group.

Item	Details
Date, room, duration	Announced on the course group (pinned message). The practice final in Section 5 is designed for 120 minutes.
Materials	Expected to be closed book, as the midterm was **[placeholder: confirm in class]**. Bring your student ID card and a pen; a pencil and an eraser help with traces and diagrams.
Calculators	Policy announced in class **[placeholder]**. No question type needs one: all arithmetic is small.
Weight	Final 60 %, midterm 40 % of the course grade.
Scope	Weeks 1–14, cumulative, with the emphasis on Weeks 9–14 (table below). Still not allowed in answers: LINQ (`.Max()`, `.Sum()`, ...) and `List<T>`.
Form	Written on paper: you write C# by hand, without a compiler or IntelliSense.

Week	Topic	Typical exam task	
1–2	Algorithms, flowcharts	pseudocode; flowchart of a loop with a decision; trace table	basis
3–4	Types, variables, operators	evaluate expressions; integer division; casts; `Parse`/`TryParse`	basis
5–7	`if`/`switch`, `for`, `while`, `do-while`	inside every program; validation loops	basis
9	One-dimensional arrays	indices, `Length`, traversal, reference vs copy	**focus**
10	Searching, sorting, max/min	trace searches and sort passes; count comparisons	**focus**
11	Matrices	nested loops, row/column sums, diagonals, grid output	**focus**
12	Methods, parameter passing	write methods; value vs `ref` vs array parameters	**focus**
13	`return`, `void`, scope, recursion	every path returns; scope errors; recursion traces	**focus**
14	Files, `try/catch`	`ReadAllLines`/`StreamReader`, `Split`, exceptions	**focus**

1.3 Question types and how to answer them

Type	Method
Multiple choice	Eliminate wrong options first. For code items, evaluate on scrap paper.
Predict the output	Draw arrays as boxes with indices; keep a table of all variables; execute one statement at a time. `string.Join` prints the elements with the separator; C# prints `True`/`False`.
Trace a sort or search	One row per pass (sort) or per step (binary search): the array after the pass, `low`/`high`/`mid`, the decision, the counters.
Trace calls	One box per call with its own parameters. Ask for every argument: copy of a value, copy of a reference (array), or the caller's variable (`ref`/`out`)? For recursion, go down to the base case, then return the values upwards.
Find the errors	Separate compile errors (scope, types, missing `return`, argument order) from run-time errors (index out of range, `FormatException`) and logic errors (wrong start value, integer division, early `return`). Always state the fix.
Write a program	Write the method headers first (return type, name, parameters), then the top-level statements that call them, then the bodies. Name the pattern. For files: create/open, loop, split, parse inside `try`, output.

Time management

Budget a little over one minute per point and keep about 10 minutes for checking. Read the whole paper first (5 minutes) and start with what you are sure of. Never leave a question blank: correct method headers, pseudocode or a partial trace usually earn partial credit.

>_ 2 Cumulative cheat sheet

This summary is for *learning*, not for the exam room. The best preparation is to write your own one-page version by hand. Weeks 1–7 are summarised briefly; the full version is in the Week 8 notes.

2.1 C# syntax summary

Basics (Weeks 3–7)

```
const double VatRate = 0.20;
int n = int.Parse(Console.ReadLine());
bool ok = int.TryParse(s, out int v); // no crash
double r = (double)7 / 2;             // 3.5
Console.WriteLine($"{r:F2} {n,5}");   // 3.50
string p = n % 2 == 0 ? "even" : "odd";
for (int i = 0; i < n; i++) { ... }
while (x != -1) { ... }               // 0 or more
do { ... } while (!ok || n < 0);      // 1 or more
```

One-dimensional arrays (Week 9)

```
int[] a = new int[5];         // 0 0 0 0 0
double[] t = { 2.5, 7, 1 };   // initializer
a[0] = 10;                    // index 0 .. Length-1
int last = a[a.Length - 1];
for (int i = 0; i < a.Length; i++)
{
    a[i] = 2 * i;             // index needed
}
foreach (double x in t)       // read-only x
{
    Console.Write(x + " ");
}
Console.WriteLine(string.Join(", ", a));
int[] same = a;               // SAME array
int[] copy = (int[])a.Clone(); // new array
string s = "kayak"; char c = s[0]; // s.Length
```

Searching and sorting (Week 10)

```
int mid = (low + high) / 2;          // binary
if (a[mid] < target) low = mid + 1;
else high = mid - 1;                 // (== : found)
if (a[j] > a[j + 1]) { swap }        // bubble
if (a[j] < a[minIndex]) minIndex = j; // selection
int t = a[i]; a[i] = a[j]; a[j] = t; // swap
(a[i], a[j]) = (a[j], a[i]);         // tuple swap
Array.Sort(a);  Array.Reverse(a);    // built-in
```

Matrices (Week 11)

```
int[,] m = new int[3, 4];        // 3 rows, 4 cols
int[,] k = { { 1, 2 }, { 3, 4 } };
int rows = m.GetLength(0);       // 3
int cols = m.GetLength(1);       // 4; m.Length 12
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        Console.Write($"{m[r, c],4}");
    }
    Console.WriteLine();
}
int[][] jag = new int[3][];      // jagged
jag[0] = new int[] { 1 };        // jag[r].Length
```

Methods (Weeks 12–13)

```
// top-level statements first, methods at the bottom
double avg = Average(scores);
Swap(ref x, ref y);
(int lo, int hi) = MinMax(scores);

// returns the mean of the elements
static double Average(int[] a)
{
    int sum = 0;
    foreach (int v in a) { sum += v; }
    return (double)sum / a.Length;
}
static void Swap(ref int p, ref int q)
{
    int t = p; p = q; q = t;
}
static (int min, int max) MinMax(int[] a)
{ ... return (min, max); }
static long Fact(int n)          // recursion
{
    if (n <= 1) { return 1; }    // base case
    return n * Fact(n - 1);
}
```

Files and exceptions (Week 14)

```
File.WriteAllLines("in.csv", lines);   // string[]
File.AppendAllText("log.txt", "x\n");
string[] all = File.ReadAllLines("in.csv");
try
{
    using (StreamReader rd = new StreamReader(path))
    {
        string line;
        while ((line = rd.ReadLine()) != null)
        {
            string[] f = line.Split(',');
            int score = int.Parse(f[1]);
        }
    }
    using (StreamWriter w = new StreamWriter(outPath))
    {
        w.WriteLine("result");       // closed at }
    }
}
catch (FileNotFoundException) { ... }
catch (FormatException) { ... }
```

2.2 Array algorithms at a glance

Pattern	Core idea	Remember
Sum / average	`sum += a[i];` then `(double)sum / a.Length`	cast before dividing
Maximum with index	`best = 0;` `if (a[i] > a[best]) best = i;`	a value maximum starts with `a[0]`, never with 0
Count if	`if (condition) count++;`	count first, then allocate a filtered array
Linear search	`if (a[i] == x) return i;` … `return -1;`	"not found" only *after* the loop; $\leq n$ comparisons
Binary search	halve `low..high` around `mid`	sorted data only; at most $\lfloor \log_2 n \rfloor + 1$ steps
Bubble sort	swap neighbours; `swapped` flag	pass k fixes the k largest; $\frac{n(n-1)}{2}$ comparisons worst case
Selection sort	index of the minimum, one swap per pass	always $\frac{n(n-1)}{2}$ comparisons, at most $n-1$ swaps
Reverse in place	swap `a[i]`, `a[n-1-i]` for `i < n / 2`	up to n instead of $n/2$ reverses twice
Frequency array	`counts[value - offset]++;`	offset 1 for dice, `'a'` for letters
Matrix rows / columns	outer loop = rows for row sums, = columns for column sums	diagonal `r == c`, anti-diagonal `r + c == n - 1`

2.3 Parameters, errors and file methods

Parameter	The method receives	Can the caller see a change?
`int v`	a copy of the value	no
`int[] a`	a copy of the reference	elements: yes; `a = new ...`: no
`ref int v`	the caller's variable itself	yes
`out int v`	the caller's variable; must be assigned	yes (that is its purpose)

Message	Typical cause (all reproduced with `dotnet build` or a real run)
`CS0103`	name does not exist in the current context: variable used outside its block
`CS0136`	a nested block declares a name that an enclosing scope already uses
`CS0128`	the same name is declared twice in one scope
`CS0161`	not all code paths return a value (missing `return` after a loop or an `if`)
`CS0165`	use of an unassigned local variable
`CS0029`	cannot convert, e.g. `int y = Show(3);` where `Show` is `void`
`CS1503`	argument has the wrong type, e.g. arguments in the wrong order
`CS8421`	a `static` local function uses a top-level variable
`CS0022 / CS1656`	`m[r][c]` on an `int[,]` / assignment to a `foreach` variable
`IndexOutOfRange`	run time: index < 0 or $\geq$ `Length` (e.g. `i <= a.Length`)
`FormatException`	run time: `int.Parse("60");` catch it or use `TryParse`
`FileNotFound`	run time: opening or reading a file that does not exist
`Stack overflow`	run time: recursion without a reachable base case (cannot be caught)

File method	Does
`File.WriteAllText(p, s) / WriteAllLines(p, arr)`	create or overwrite
`File.AppendAllText(p, s)`	add at the end (creates the file if needed)
`File.ReadAllText(p) / ReadAllLines(p)`	whole text / `string[]`, one element per line
`File.Exists(p) / File.Delete(p)`	test / remove
`new StreamReader(p).ReadLine()`	next line, `null` at the end of the file
`new StreamWriter(p).WriteLine(s)`	overwrites; inside `using`, or the file may stay empty

Relative paths such as `"grades.csv"` refer to the working directory; with `dotnet run` that is the project folder. Pseudocode for arrays uses the same conventions as before: **FOR** $i \leftarrow 0$ **TO** $n-1$, $a[i] \leftarrow 0$, and $\leftarrow$ assigns while $=$ compares.

>_ 3 Common mistakes

Ten mistakes that cost points in the second half

1. `i <= a.Length` in a loop: the last pass uses index `Length` → `IndexOutOfRangeException`. Use `i < a.Length`.
2. `int max = 0;` for a value maximum: wrong when all values are negative. Start with `a[0]`.
3. `int[] copy = a;` is not a copy: both names refer to one array. Use `(int[])a.Clone()` or `new` and a loop.
4. `m.Length` as the number of rows: it is the number of *all* elements. Use `m.GetLength(0)` and `m.GetLength(1)`.
5. `m[r][c]` on a rectangular array: CS0022. Write `m[r, c]`.
6. `else { return false; }` inside a search loop decides after the first element. Return "not found" after the loop.
7. A value-returning method without a final `return`: CS0161, "not all code paths return a value".
8. Changing a value parameter and expecting the caller to see it. Return the new value, or use `ref`.
9. A `StreamWriter` without `using` (or `Close`): the buffer may never be written; in our test the file stayed empty (0 bytes).
10. `int.Parse` of file data without `try/catch`: one bad line (`60` instead of `60`) stops the whole program with a `FormatException`.

>_ 4 Review questions from the lecture

These are the fourteen practice questions of the Week 15 slides with worked answers. Cover the answer, solve the question on paper, then compare. All predict and trace outputs were produced by `15_01_PredictChecker`; the compile errors were produced by building the code with `dotnet build`.

4.1 Q1–Q4: multiple choice

Q1. `double[] t = new double[4];`
What does `Console.WriteLine(t[4]);` do? (A) prints 0 (B) prints 0.0 (C) prints nothing (D) throws an exception

Q2. `int[,] m = new int[3, 5];` What is `m.GetLength(1)`? (A) 3 (B) 5 (C) 15 (D) 2

Q3. Which call lets a method change the caller's `int` variable `x`? (A) `Change(x)` (B) `Change(ref`

`x)` (C) `Change(x + 1)` (D) `Change((int)x)`

Q4. What is the return type of `File.ReadAllLines(path)`? (A) `string` (B) `string[]` (C) `char[]` (D) `void`

Answers

Q1 (D): valid indices are 0–3; the code compiles but throws `IndexOutOfRangeException` ("Index was outside the bounds of the array."). Q2 (B): dimension 1 is the columns. Q3 (B): only `ref` passes the variable itself; the method header must also say `ref int`. Q4 (B): one string per line.

4.2 Q5: arrays

```
1  int[] a = { 4, 7, 1, 9, 3 };
2  int[] b = a;
3  b[0] = 10;
4  Console.WriteLine(a[0]);
5  Console.WriteLine(a[a.Length - 1]);
6  Console.WriteLine(a[a[2]]);
7  int sum = 0;
8  for (int i = 1; i < a.Length; i += 2)
9  {
10     sum += a[i];
11 }
12 Console.WriteLine(sum);
13 int[] c = (int[])a.Clone();
14 c[1] = 0;
15 Console.WriteLine(string.Join(",", a));
16 Console.WriteLine(a == c);
```

Output

```
10
3
7
16
10,7,1,9,3
False
```

Explanation. `b = a` gives the same array a second name, so `b[0] = 10` changes `a[0]`. The last index is Length $-1 = 4$. `a[2]` is 1, so `a[a[2]]` is `a[1]` $= 7$. The loop visits $i = 1, 3$: $7 + 9 = 16$. `Clone` creates a new array, so changing `c` does not change `a`, and `==` on arrays compares references: `False`.

4.3 Q6: bubble sort, pass by pass

```
int[] a = { 4, 2, 7, 1, 9, 5 };
int n = a.Length;
for (int pass = 1; pass <= n - 1; pass++)
{
    bool swapped = false;
    for (int j = 0; j < n - pass; j++)
    {
        if (a[j] > a[j + 1])
        {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
            swapped = true;
        }
    }
    if (!swapped)
    {
        break;
    }
}
```

pass	array after the pass	comp.	swaps
start	4 2 7 1 9 5	–	–
1	2 4 1 7 5 9	5	3
2	2 1 4 5 7 9	4	2
3	1 2 4 5 7 9	3	1
4	1 2 4 5 7 9	2	0
total		14	6

After pass k the k largest values are in their final places. Pass 3 already sorts the array, but only pass 4 (no swap) lets the algorithm *detect* it; `break` then skips pass 5. Without the early exit there would be $5+4+3+2+1 = 15 = \frac{6\cdot5}{2}$ comparisons.

4.4 Q7: binary search trace

Array `a = { 3, 8, 11, 15, 19, 24, 30, 36, 41, 47, 52 }` (indices 0–10), code as in Week 10: `mid = (low + high) / 2`; equal: found; `a[mid] < target`: `low = mid + 1`; otherwise `high = mid - 1`; loop while `low <= high` and not found.

target	step	low	high	mid	a[mid]	decision
47	1	0	10	5	24	$24 < 47$: `low = 6`
	2	6	10	8	41	$41 < 47$: `low = 9`
	3	9	10	9	47	found, index 9
12	1	0	10	5	24	$24 > 12$: `high = 4`
	2	0	4	2	11	$11 < 12$: `low = 3`
	3	3	4	3	15	$15 > 12$: `high = 2`
		3	2			$low > high$: not found (-1)

Both searches need 3 steps; a linear search needs 10 comparisons for 47 and 11 for 12.

4.5 Q8: nested loops on a matrix

```
int[,] m = { { 2, 7, 1 }, { 5, 3, 8 }, { 4, 6, 9 } };
int n = m.GetLength(0);
for (int r = 0; r < n; r++)
{
    for (int c = 0; c < n; c++)
    {
        if (r > c)
        {
            Console.Write("  .");
        }
        else
        {
            Console.Write($"{m[r, c],3}");
        }
    }
    Console.WriteLine();
}
int diag = 0, anti = 0;
for (int i = 0; i < n; i++)
{
    diag += m[i, i];
    anti += m[i, n - 1 - i];
}
Console.WriteLine($"{diag} {anti}");
```

Output

```
  2  7  1
  .  3  8
  .  .  9
14 8
```

Cells with $r > c$ lie below the main diagonal and print `" ."`. `diag` adds $(0,0), (1,1), (2,2)$: $2+3+9 = 14$; `anti` adds $(0,2), (1,1), (2,0)$: $1+3+4 = 8$.

4.6 Q9: value, `ref` and array parameters

```
int x = 5, y = 5;
int[] arr = { 5, 5 };
Bump(x);
BumpRef(ref y);
BumpArray(arr);
Replace(arr);
Console.WriteLine($"{x} {y} {arr[0]} {arr[1]}");
int z = Twice(x) + Twice(Twice(1));
Console.WriteLine(z);
```

Output

```
5 6 6 5
14
```

```
static void Bump(int v)
{
    v++;
}
static void BumpRef(ref int v)
{
    v++;
}
static void BumpArray(int[] a)
{
    a[0]++;
}
static void Replace(int[] a)
{
    a = new int[] { 0, 0 };
    a[1] = 99;
}
static int Twice(int v)
{
    return 2 * v;
}
```

Explanation. `Bump` changes its own copy: $x = 5$. `BumpRef`: `v` *is* `y`, so $y = 6$. `BumpArray` receives a copy of the reference, so it changes the caller's element: `arr[0]` $= 6$. `Replace` only redirects its local copy of the reference to a new array; the caller's array is unchanged. Twice$(5) = 10$ and Twice(Twice(1)) = Twice$(2) = 4$, so $z = 14$.

4.7 Q10: recursion

```
static int Mystery(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return n % 2 + 10 * Mystery(n / 2);
}
static void Countdown(int n)
{
    if (n == 0)
    {
        return;
    }
    Console.Write(n + " ");
    Countdown(n - 1);
    Console.Write(n + " ");
}
```

(a) `Mystery(6)`: the calls go down to the base case, the values come back up:

call	pending expression	returns
`Mystery(6)`	$0 + 10 \cdot \text{Mystery}(3)$	110
`Mystery(3)`	$1 + 10 \cdot \text{Mystery}(1)$	11
`Mystery(1)`	$1 + 10 \cdot \text{Mystery}(0)$	1
`Mystery(0)`	base case	0

Four calls; the result `110` is 6 written in binary (the Week 7 conversion, recursively).

(b) `Countdown(3)` prints `3 2 1 1 2 3`: each call writes n, calls itself, and writes n again after the inner call has returned.

4.8 Q11: spot the compile errors

```
int total = 0;
for (int i = 1; i <= 3; i++)
{
    int square = i * i;
    total += square;
}
Console.WriteLine(square);
if (total > 10)
{
    int total = 0;
    string msg = "big";
}
else
{
    string msg = "small";
}
int count;
Console.WriteLine(count + total);
Console.WriteLine(Half(total));

static int Half(int n)
{
    if (n >= 0)
    {
        return n / 2;
    }
}
```

line	error	reason
7	`CS0103`	`square` exists only inside the loop block
10	`CS0136`	nested block reuses the name `total`
18	`CS0165`	`count` is never assigned
21	`CS0161`	no `return` when $n < 0$

Lines 11 and 15 are *correct*: two sibling blocks may each declare their own `msg` (the compiler only warns CS0219, "assigned but never used"). Fixes: use `square` inside the loop; rename the inner `total`; write `int count = 0;`; add a final `return` (e.g. `return 0;`) to `Half`.

4.9 Q12: find the bugs

Intended: read 5 daily temperatures (whole degrees, possibly negative) into an array; print the average with 2 decimals and the highest temperature.

```
1  int[] temps = new int[5];
2  for (int i = 1; i <= temps.Length; i++)
3  {
4      Console.Write($"Day {i}: ");
5      temps[i] = int.Parse(Console.ReadLine());
6  }
7  Console.WriteLine($"Average: {Average(temps):F2}")
       ;
8  Console.WriteLine($"Highest: {Highest(temps)}");
9
10 static double Average(int[] a)
11 {
12     int sum = 0;
13     foreach (int t in a)
14     {
15         sum += t;
16     }
17     return sum / a.Length;
18 }
```

```
20 static int Highest(int[] a)
21 {
22     int max = 0;
23     for (int i = 0; i < a.Length; i++)
24     {
25         if (a[i] > max)
26         {
27             max = i;
28         }
29     }
30     return max;
31 }
```

line	bug	fix
2	indices 1–5: `a[0]` is never filled and `temps[5]` crashes	`for (int i = 0; i < temps.Length; i++)`
17	`sum / a.Length` is integer division	`return (double)sum / a.Length;`
22	starting with 0 fails when all temperatures are negative	`int max = a[0];`
27	stores the index instead of the value	`max = a[i];`

Real run of the buggy program with $-3, -7, -2, -1, -4$: all five values are read, then the store into `temps[5]` throws `IndexOutOfRangeException` (line 5). With only the loop fixed it prints `Average: -3.00` and `Highest: 0`; with $3, 7, 2, 1, 4$ the buggy `Highest` returns 4, an index. The corrected program:

Listing 1: `15_02_TempStatsFixed`

```
1  // MCMP113 · Week 15 · Example 2 --- Review Q12: the "find the bugs" temperature program, corrected
2  // Read 5 daily temperatures into an array; print the average and the highest with two methods.
3
4  int[] temps = new int[5];
5  for (int i = 0; i < temps.Length; i++)          // fix 1: indices 0 .. Length - 1
6  {
7      Console.Write($"Day {i + 1}: ");
8      temps[i] = int.Parse(Console.ReadLine());
9  }
10 Console.WriteLine($"Average: {Average(temps):F2}");
11 Console.WriteLine($"Highest: {Highest(temps)}");
12
13 // Average of the elements (real division)
14 static double Average(int[] a)
15 {
16     int sum = 0;
17     foreach (int t in a)
18     {
```

```
        sum += t;
    }
    return (double)sum / a.Length;              // fix 2: no integer division
}

// Largest element (best-so-far pattern)
static int Highest(int[] a)
{
    int max = a[0];                             // fix 3: start with the first element, not 0
    for (int i = 1; i < a.Length; i++)
    {
        if (a[i] > max)
        {
            max = a[i];                         // fix 4: store the value, not the index
        }
    }
    return max;
}
```

Output

```
Day 1: -3
Day 2: -7
Day 3: -2
Day 4: -1
Day 5: -4
Average: -3.40
Highest: -1
```

4.10 Q13: write the methods

The top-level statements and `ReadSteps` *are given. Write* `Average`, `IndexOfMax` *and* `CountAbove` *so that the program prints the average, the best day and the number of days above the average.* `IndexOfMax` returns an *index*, so the caller can print both `days[best]` and `steps[best]`; `CountAbove` takes a `double` limit because the average is a `double`.

Listing 2: `15_03_WeeklySteps`

```
// MCMP113 · Week 15 · Example 3 --- Review Q13: write the methods (weekly step counter)
// Arrays + methods: read 7 values with validation, then average, best day and days above average.

string[] days = { "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun" };
int[] steps = ReadSteps(days);

double average = Average(steps);
int best = IndexOfMax(steps);
int above = CountAbove(steps, average);
Console.WriteLine($"Average: {average:F1} steps");
Console.WriteLine($"Best day: {days[best]} ({steps[best]} steps)");
Console.WriteLine($"Days above average: {above}");

// Reads one non-negative value per day; repeats the prompt until the input is valid
static int[] ReadSteps(string[] dayNames)
{
    int[] values = new int[dayNames.Length];
    for (int i = 0; i < values.Length; i++)
    {
        int v;
        bool ok;
        do
        {
```

```
            Console.Write($"{dayNames[i]}: ");
            ok = int.TryParse(Console.ReadLine(), out v) && v >= 0;
            if (!ok)
            {
                Console.WriteLine("  Please enter a whole number >= 0.");
            }
        } while (!ok);
        values[i] = v;
    }
    return values;
}

// Average of the elements
static double Average(int[] a)
{
    int sum = 0;
    foreach (int x in a)
    {
        sum += x;
    }
    return (double)sum / a.Length;
}

// Index of the first largest element
static int IndexOfMax(int[] a)
{
    int best = 0;
    for (int i = 1; i < a.Length; i++)
    {
        if (a[i] > a[best])
        {
            best = i;
        }
    }
    return best;
}

// How many elements are strictly greater than limit
static int CountAbove(int[] a, double limit)
{
    int count = 0;
    foreach (int x in a)
    {
        if (x > limit)
        {
            count++;
        }
    }
    return count;
}
```

Output

```
Mon: 6500
Tue: 8200
Wed: abc
  Please enter a whole number >= 0.
Wed: 7400
Thu: 12050
Fri: 9900
Sat: 3100
Sun: 10400
```

```
Average: 8221.4 steps
Best day: Thu (12050 steps)
Days above average: 3
```

Check: $6500 + 8200 + 7400 + 12050 + 9900 + 3100 + 10400 = 57\,550$, $57\,550/7 \approx 8221.4$; above it: Thu, Fri, Sun.

4.11 Q14: a CSV file program

Call the given `CreateSampleFile("grades.csv")`. Read the file with a `StreamReader` inside `using` and skip the header. Split each line at commas; a line without 3 fields or with a bad number is skipped with a message. Total $= 0.4 \cdot$ *midterm* $+ 0.6 \cdot$ *final; write name, total (2 decimals) and PASS/FAIL* (≥ 50) *to `report.txt`. Print the number of students, the class average and the report; catch `FileNotFoundException`.*

Design. The counters are declared *before* the `try` because they are needed after it. One `ReadLine()` before the loop skips the header; `(line = reader.ReadLine()) != null` reads and tests in one expression. The field-count check uses `continue`; the parsing (inside `Total`) sits in its own `try/catch`, so one bad line does not stop the loop.

Listing 3: `15_04_GradeReportCsv`

```
// MCMP113 · Week 15 · Example 4 --- Review Q14: read a CSV file, write a report file
// The program first creates grades.csv itself, then reads it line by line with StreamReader.

const string InputFile = "grades.csv";
const string ReportFile = "report.txt";
CreateSampleFile(InputFile);

int count = 0;
double sum = 0;
try
{
    using (StreamReader reader = new StreamReader(InputFile))
    using (StreamWriter writer = new StreamWriter(ReportFile))
    {
        reader.ReadLine();                      // skip the header line
        string line;
        while ((line = reader.ReadLine()) != null)
        {
            string[] parts = line.Split(',');
            if (parts.Length != 3)
            {
                Console.WriteLine($"Skipped (3 fields expected): {line}");
                continue;
            }
            try
            {
                double total = Total(parts[1], parts[2]);
                string result = total >= 50 ? "PASS" : "FAIL";
                writer.WriteLine($"{parts[0],-8}{total,7:F2}  {result}");
                count++;
                sum += total;
            }
            catch (FormatException)
            {
                Console.WriteLine($"Skipped (not a number): {line}");
            }
        }
    }
}
catch (FileNotFoundException)
```

```
{
    Console.WriteLine($"File not found: {InputFile}");
}
PrintSummary(count, sum, ReportFile);

// Writes the sample data (header + one line per student) into path
static void CreateSampleFile(string path)
{
    string[] lines =
    {
        "name,midterm,final",
        "Ayşe,72,85",
        "Mehmet,45,58",
        "Zeynep,90,94",
        "Can,6O,70",
        "Deniz,80",
        "Elif,38,51",
    };
    File.WriteAllLines(path, lines);
}

// Course total: 40 % midterm + 60 % final; int.Parse throws FormatException for a bad number
static double Total(string mid, string fin)
{
    return 0.4 * int.Parse(mid)
         + 0.6 * int.Parse(fin);
}

// Prints the number of students, the class average and the report file
static void PrintSummary(int count, double sum, string path)
{
    if (count == 0)
    {
        Console.WriteLine("No valid lines.");
        return;
    }
    double avg = sum / count;
    Console.WriteLine($"Students: {count}");
    Console.WriteLine($"Average: {avg:F2}");
    Console.WriteLine($"--- {path} ---");
    Console.Write(File.ReadAllText(path));
}
```

Output

```
Skipped (not a number): Can,6O,70
Skipped (3 fields expected): Deniz,80
Students: 4
Average: 67.70
--- report.txt ---
Ayşe      79.80  PASS
Mehmet    52.80  PASS
Zeynep    92.40  PASS
Elif      45.80  FAIL
```

Check: $0.4 \cdot 72 + 0.6 \cdot 85 = 79.8$, $0.4 \cdot 45 + 0.6 \cdot 58 = 52.8$, $0.4 \cdot 90 + 0.6 \cdot 94 = 92.4$, $0.4 \cdot 38 + 0.6 \cdot 51 = 45.8$; average $270.8/4 = 67.7$. The 50-point pass mark is an example; the official rules are in the university regulations.

>_ 5 Practice Final Exam

MCMP113 · Practice Final Exam Duration: 120 minutes · Total: 100 points

Name: _____ Student no.: _____

Instructions. Answer on the question paper where space is given and on separate answer sheets otherwise. Write C# with top-level statements; methods are `static` local functions written after the top-level statements. Do not use LINQ or `List<T>`. Show your work: partial credit is given for correct reasoning, method headers, pseudocode and partial traces. (Materials and calculator rules of the real exam: see the course group.)

Question	1	2	3	4	5	6	7	8	Total
Topic	MC	Predict	Sort	Matrix	Methods	Bugs	Arrays	Files	
Points	20	15	10	12	13	10	10	10	100
Score									

Question 1 – Multiple choice [20 pts]

Circle one answer per item (2 points each).

1.1 After `int[] a = new int[n];` the index of the last element is
(a) `n` (b) `n - 1` (c) `n + 1` (d) `a.Length`

1.2 After `bool[] flags = new bool[3];` every element is
(a) `null` (b) `0` (c) `false` (d) `true`

1.3 Binary search can only be used when the array is
(a) free of duplicates (b) sorted (c) of even length (d) an `int[]`

1.4 How many comparisons does bubble sort *without* early exit make for 6 elements?
(a) 6 (b) 15 (c) 30 (d) 36

1.5 After `int[,] m = new int[2, 3];` the value of `m.Length` is
(a) 2 (b) 3 (c) 5 (d) 6

1.6 Which header fits the call `double r = Avg(a);` where `a` is an `int[]`?
(a) `static void Avg(int[] a)` (b) `static double Avg(int a)`
(c) `static double Avg(int[] a)` (d) `static int[] Avg(double a)`

1.7 For `static void Show(int x)`, which statement does *not* compile?
(a) `Show(3);` (b) `Show(2 + 1);` (c) `int y = Show(3);` (d) `Show(int.Parse("3"));`

1.8 A recursive method whose base case is never reached (a) returns 0 (b) is a compile error (c) ends with a stack overflow at run time (d) runs forever without any error

1.9 After `string[] parts = "Ali,85,90".Split(',');` the values of `parts.Length` and `parts[1]` are
(a) 3 and `"85"` (b) 3 and `"Ali"` (c) 2 and `"85"` (d) 9 and `"l"`

1.10 What does `using (StreamWriter w = new StreamWriter("a.txt")) { ... }` guarantee?
(a) the file is only created if it does not exist (b) the file is closed (and flushed) at the end of the block, even after an exception (c) no exception can occur (d) the text is appended to the file

Question 2 – Predict the output [15 pts]

Write the exact output of each fragment (3 points each). Each fragment runs on its own; methods are written at the end of the program.

(a)

```
int[] a = { 2, 4, 6, 8, 10 };
for (int i = 0; i < a.Length; i++)
{
    int t = a[i];
    a[i] = a[a.Length - 1 - i];
    a[a.Length - 1 - i] = t;
}
Console.WriteLine(string.Join(" ", a));
```

(c)

```
int[] data = { 3, 1, 3, 0, 2, 3, 1 };
int[] freq = new int[4];
foreach (int d in data)
{
    freq[d]++;
}
Console.WriteLine(string.Join(",", freq));
```

(b)

```
string w = "level up";
int count = 0;
for (int i = 0; i < w.Length; i++)
{
    if (w[i] == 'e' || w[i] == 'u')
    {
        count++;
    }
}
Console.WriteLine($"{w.Length} {count} {w[w.Length - 1]}");
```

(d)

```
(int lo, int hi) = MinMax(new int[] { 7, -2, 9, 4 });
Console.WriteLine($"{lo} {hi}");
Console.WriteLine(Clamp(15, 0, 10) + Clamp(-3, 0, 10));

// given (Week 13, Example 4):
// static (int min, int max) MinMax(int[] a)
static int Clamp(int v, int lo, int hi)
{
    if (v < lo)
    {
        return lo;
    }
    if (v > hi)
    {
        return hi;
    }
    return v;
}
```

(e)

```
File.WriteAllText("t.txt", "a\nb\n");
File.AppendAllText("t.txt", "c\n");
string[] lines = File.ReadAllLines("t.txt");
Console.WriteLine($"{lines.Length} {lines[lines.Length - 1]}");
File.WriteAllLines("t.txt", new string[] { "x" });
Console.WriteLine(File.ReadAllLines("t.txt").Length);
```

Question 3 – Trace a sort [10 pts]

```
int[] a = { 29, 10, 14, 37, 13 };
int n = a.Length;
for (int i = 0; i < n - 1; i++)
{
    int minIndex = i;
    for (int j = i + 1; j < n; j++)
    {
        if (a[j] < a[minIndex])
        {
            minIndex = j;
        }
    }
    if (minIndex != i)
    {
        int temp = a[i];
        a[i] = a[minIndex];
        a[minIndex] = temp;
    }
}
```

(a) Complete the table: the value of `minIndex` at the end of each pass and the array *after* the pass. (8)

pass (i)	`minIndex`	`array`
start	–	`29 10 14 37 13`
0		
1		
2		
3		

(b) How many comparisons `a[j] < a[minIndex]` and how many swaps are made in total? (2)

Question 4 – Matrices [12 pts]

(a) Write the four output lines of this fragment. (4)

```
int rows = 3, cols = 4;
int[,] g = new int[rows, cols];
for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        g[r, c] = r * cols + c;
    }
}
Console.WriteLine(g[2, 1]);
Console.WriteLine(g.Length);
for (int c = 0; c < cols; c++)
{
    Console.Write(g[1, c] % 3 + " ");
}
Console.WriteLine();
int s = 0;
for (int r = 0; r < rows; r++)
{
    s += g[r, cols - 1];
}
Console.WriteLine(s);
```

(b) A company has 3 shops and records the sales of 4 quarters in `int[,] sales` (row = shop, column = quarter). Write the code that prints the total of each shop (`Shop 1: 465`, …) and then the best quarter, i.e. the column with the largest total (`Best quarter: Q4 (358)`). Use `GetLength`; do not assume the sizes. (8)

```
int[,] sales =
{
    { 120, 95, 140, 110 },
    { 80, 105, 90, 150 },
    { 150, 120, 100, 98 },
};
```

Question 5 – Methods, parameters and scope [13 pts]

(a) Write the three output lines. (5)

```
int a = 3;
int[] v = { 1, 2, 3 };
Change(a, v);
Console.WriteLine($"{a} {string.Join(",", v)}");
Swap(ref v[0], ref v[2]);
Console.WriteLine(string.Join(",", v));
int b = Next(ref a) + Next(ref a);
Console.WriteLine($"{a} {b}");

static void Change(int x, int[] arr)
{
    x = x * 10;
    arr[0] = x;
    arr = new int[] { 0, 0, 0 };
    arr[1] = 99;
}
static void Swap(ref int p, ref int q)
{
    int t = p;
    p = q;
    q = t;
}
static int Next(ref int n)
{
    n++;
    return n;
}
```

(b) The following program does not compile. Give the line number and the reason of each of the **four** errors. (5)

```
int limit = 5;
int sum = Total(limit);
Console.WriteLine(result);
for (int k = 0; k < 2; k++)
{
    int limit = k;
}
Show(sum);
int shown = Show(sum);

static int Total(int n)
{
    int result = 0;
    for (int i = 1; i <= n; i++)
    {
        result += i;
    }
    return result;
}

static void Show(int value)
{
    Console.WriteLine(value + limit);
}
```

(c) For the method below, what is `G(4, 3)`, and how many calls of `G` does it make in total? What happens for `G(4, -1)`? (3)

```
static int G(int a, int b)
{
    if (b == 0)
    {
        return 0;
    }
    return a + G(a, b - 1);
}
```

Question 6 – Find and fix the errors [10 pts]

The program should print how many numbers are even, whether 7 occurs, and a doubled *copy* of the array while the original stays unchanged. It contains **five** errors. (a) Find each error (line number), explain it and give the fix. (b) Which of them are compile errors?

```
int[] numbers = { 4, 7, 10, 3, 8 };
Console.WriteLine($"Even: {CountEven(numbers)}");
Console.WriteLine($"Contains 7: {Contains(7,
    numbers)}");
int[] doubled = Doubled(numbers);
Console.WriteLine("Original: " + string.Join(" ",
    numbers));
Console.WriteLine("Doubled:  " + string.Join(" ",
    doubled));

static int CountEven(int[] a)
{
    int count = 0;
    for (int i = 0; i <= a.Length; i++)
    {
        if (a[i] % 2 == 0)
        {
            count++;
        }
        return count;
    }
}
```

```
static bool Contains(int[] a, int target)
{
    for (int i = 0; i < a.Length; i++)
    {
        if (a[i] == target)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    return false;
}

static int[] Doubled(int[] a)
{
    int[] result = a;
    for (int i = 0; i < a.Length; i++)
    {
        result[i] = a[i] * 2;
    }
    return result;
}
```

Question 7 – Write a program: arrays and methods [10 pts]

Write a program that reads the number of days n ($1 \le n \le 31$; ask again until the input is valid) and then n temperatures (real numbers) into an array. It prints the average (2 decimals), the hottest day (day number counted from 1 and its value with 1 decimal), the number of days above the average, the temperatures sorted in ascending order, and the original order. Write and use at least these methods: `ReadIntInRange(prompt, min, max)`, `ReadArray(n)`, `Average`, `IndexOfMax`, `CountAbove(a, limit)` and `SortedCopy(a)`, which returns a sorted *copy* with your own sorting loop (no `Array.Sort`); the original array must stay unchanged. Sample run:

Output

```
Number of days (1-31): 0
Number of days (1-31): 6
Day 1: 14.5
Day 2: 18
Day 3: 9.5
Day 4: 21
Day 5: 17.5
Day 6: 12
Average: 15.42
Hottest: day 4 (21.0)
Days above average: 3
Sorted:   9.5 12 14.5 17.5 18 21
Original: 14.5 18 9.5 21 17.5 12
```

Question 8 – Write a program: files [10 pts]

(a) Write `CreateSalesFile(path)`, which writes these six lines with `File.WriteAllLines`: `product,quantity,unitPrice` / `Keyboard,3,450.00` / `Mouse,two,150.00` / `Monitor,2,3250.50` / `USB cable,10,39.90` / `Headset,1,899.99`.

(b) Write `Summarise(inPath, outPath)`: read all lines with `File.ReadAllLines`, skip the header, split each line at commas, convert the quantity (`int`) and the unit price (`decimal`). A line with a bad number prints `Skipped line k: ...` (k = line number in the file, counted from 1) and is ignored. Compute the number of items sold, the revenue and the best product (largest line total). Write three lines to `outPath` with a `StreamWriter` inside `using`, then print the file. If `inPath` does not exist, print `Error: <name> was not found.`

(c) Call `CreateSalesFile("sales.csv")`. Then call `Summarise` for `sales.csv` and, as a test, for `missing.csv`.

— End of the practice final —

>_ 6 Additional problem set

Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging. For programming problems, write the method headers and pseudocode first; the tested solutions are in the code folder (projects `15_09`–`15_15`). Trace answers are printed by `15_01_PredictChecker`.

P1. ⋆ **Indices.** What does this fragment print?

```
int[] a = new int[6];
for (int i = 0; i < a.Length; i++)
{
    a[i] = i * i % 5;
}
Console.WriteLine(string.Join(" ", a));
int k = 0;
while (k < a.Length && a[k] != 4)
{
    k++;
}
Console.WriteLine(k);
```

P2. ⋆ **Linear search by hand.** In `{ 12, 5, 8, 21, 5, 30 }`, give for the targets 5, 21 and 7: the first index (or -1), the number of comparisons of a linear search that stops at the first match, and the number of occurrences.

P3. ⋆ **Column by column.** What does this print for `int[,] m = { { 1, 2, 3 }, { 4, 5, 6 } };`?

```
for (int c = 0; c < m.GetLength(1); c++)
{
    for (int r = m.GetLength(0) - 1; r >= 0; r--)
    {
        Console.Write(m[r, c] + " ");
    }
}
```

P4. ⋆⋆ **How many steps?** What is the largest number of steps a binary search needs for a sorted array of $n = 15$, 100, 1000 and 10^6 elements? Compare with linear search.

P5. ⋆⋆ **Recursion trace.** What do `PrintBinary(13)`, `Digits(2026)` and `Digits(7)` produce?

```
static void PrintBinary(int n)
{
    if (n > 0)
    {
        PrintBinary(n / 2);
        Console.Write(n % 2);
    }
}
static int Digits(int n)
{
    if (n < 10)
    {
        return 1;
    }
    return 1 + Digits(n / 10);
}
```

P6. ⋆ **Remove a value.** Write `static int[] RemoveAll(int[] a, int x)`, which returns a *new* array that contains all elements of `a` except those equal to `x`, in the same order. Hint: count first, then allocate and fill.

P7. ⋆⋆ **Longest run.** Write `static (int value, int length) LongestRun(int[] a)`, which returns the value and the length of the longest block of equal neighbouring elements (the first one if there is a tie). For `{ 3, 5, 5, 2, 5, 5, 5, 1, 1 }` the answer is $(5, 3)$.

P8. ⋆⋆ **Magic square.** A square matrix is *magic* if every row, every column and both diagonals have the same sum. Write `RowSum(m, r)`, `ColumnSum(m, c)` and `bool IsMagic(int[,] m)` and test them with the Lo Shu square `{ {2,7,6}, {9,5,1}, {4,3,8} }`, the same square with 8 and 3 swapped in the last row, and Dürer's 4×4 square.

P9. ⋆⋆ **Letter frequency.** Read a line of text and count the letters `a`–`z` (ignoring case) in an `int[26]` array with the index `ch - 'a'`. Print every letter that occurs with its count and a bar of stars, the most frequent letter, and how many other letters (for example ü, ş) occurred.

P10. ⋆⋆ **Ranking.** The parallel arrays
`names = { "Mert", "Ece", "Ali", "Selin", "Burak", "Deniz" }` and
`scores = { 78, 91, 78, 85, 91, 60 }` belong together. Sort both with selection sort: higher score first, equal scores alphabetically by name. Print a ranking in which equal scores share a rank ($1, 1, 3, \ldots$).

P11. ⋆⋆ **Log filter.** Create `app.log` with eight lines of the form `2026-12-14 09:21 ERROR Cannot open grades.csv`. Read it with a `StreamReader`, count the lines per level (`INFO`, `WARN`, `ERROR`) with two parallel arrays, and copy the `ERROR` lines with their line numbers into `errors.txt` with a `StreamWriter`. Print the counts and `errors.txt`.

P12. ⋆⋆⋆ **Grade book from a file.** Create `gradebook.csv` with a header `name,exam1,exam2,exam3` and five students. Read it with `File.ReadAllLines` into a `string[] names` and a `double[,] scores` whose size follows from the file. Print the average of every exam (column averages), compute the average of every student (row averages), sort students and averages together (highest first) and write the ranking to `ranking.txt`. Handle a missing file and a bad number.

P13. ⋆ **Copy or reference?** With `AddTwice(int v) { v = v + v; return v; }`, `Grow(int v) { v *= 3; }` and `GrowRef(ref int v) { v *= 3; }` (written with braces on their own lines in a real program), what is printed?

```
int n = 1;
for (int i = 0; i < 3; i++)
{
    n = AddTwice(n);
}
Console.WriteLine(n);
int m = 1;
Grow(m);
GrowRef(ref m);
GrowRef(ref m);
Console.WriteLine(m);
```

P14. ⋆⋆ **Recursion practice.** Write recursive methods `long Power(long b, int e)` for $e \geq 0$, `int SumArray(int[] a, int n)` (sum of the first n elements) and `bool IsPalindrome(string s, int lo, int hi)`. Name the base case of each, and test the methods with PASS/FAIL self-tests as in Week 13.

>_ 7 Summary

1. The final is cumulative (Weeks 1–14); most points come from arrays, searching and sorting, matrices, methods, scope, recursion and files.
2. Arrays: indices $0 \ldots \text{Length} - 1$; `b = a` is a second name, not a copy; a value maximum starts with `a[0]`.
3. Trace sorts and searches row by row: the array after each pass, `low`/`high`/`mid`, comparisons and swaps.
4. Matrices: `m[r, c]`, `GetLength(0)` rows, `GetLength(1)` columns; the outer loop decides whether you work row by row or column by column.
5. Arguments are copied: a value parameter never changes the caller, `ref`/`out` do, and an array parameter shares its elements.
6. Every path of a value-returning method returns; a variable lives until the `}` of its block; recursion needs a reachable base case.
7. Files: `using`, `ReadLine()` until `null` (or `ReadAllLines`), `Split`, `Parse` inside `try`/`catch`.

Glossary (English – Turkish)

final exam	final sınavı	cumulative	kümülatif (birikimli)
practice exam	deneme sınavı	rubric	puanlama anahtarı
array	dizi	index	indis (sıra numarası)
element	eleman	reference type	başvuru (referans) türü
linear search	doğrusal arama	binary search	ikili arama
bubble sort	kabarcık sıralaması	selection sort	seçmeli sıralama
comparison	karşılaştırma	swap	yer değiştirme
matrix	matris	row / column	satır / sütun
main diagonal	ana köşegen	jagged array	düzensiz (çentikli) dizi
method (function)	metot (fonksiyon)	parameter / argument	parametre / argüman
pass by value	değer ile aktarım	pass by reference	başvuru ile aktarım
return value	dönüş değeri	scope	kapsam (geçerlilik alanı)
recursion	özyineleme	base case	temel durum
call stack	çağrı yığını	stack overflow	yığın taşması
file / path	dosya / yol	stream	akış
exception	istisna (özel durum)	header line	başlık satırı
compile-time error	derleme zamanı hatası	run-time error	çalışma zamanı hatası

References and further reading

[1] Microsoft, *Arrays (C# reference)*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays

[2] Microsoft, *Array.Sort method*. https://learn.microsoft.com/en-us/dotnet/api/system.array.sort

[3] Microsoft, *Methods (C# programming guide)*. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/methods

[4] Microsoft, *Method parameters and modifiers* (`ref`, `out`). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters

[5] Microsoft, *Local functions*. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/local-functions

[6] Microsoft, *Tuple types*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-tuples

[7] Microsoft, *File class*. https://learn.microsoft.com/en-us/dotnet/api/system.io.file

[8] Microsoft, *StreamReader class*. https://learn.microsoft.com/en-us/dotnet/api/system.io.streamreader

[9] Microsoft, *using statement*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/using

[10] Microsoft, *Exception-handling statements: throw and try, catch, finally*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements

[11] Microsoft, *String.Split method*. https://learn.microsoft.com/en-us/dotnet/api/system.string.split

[12] Microsoft, *C# compiler errors and warnings*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/compiler-messages/

[13] MCMP113 Week 1–14 lecture notes and code folders; Week 8 notes for the first-half cheat sheet and practice midterm.